

Course 1 -- Java基础语法速通

本来Course1想直接讲类和对象，但还是觉得先粗略地讲解一下Java的基础语法

本章内容并没有涉及所有的java语句用法，只是择其重点，且内容比较杂，建议自己到网上寻找更多详细教程

第一个Java程序

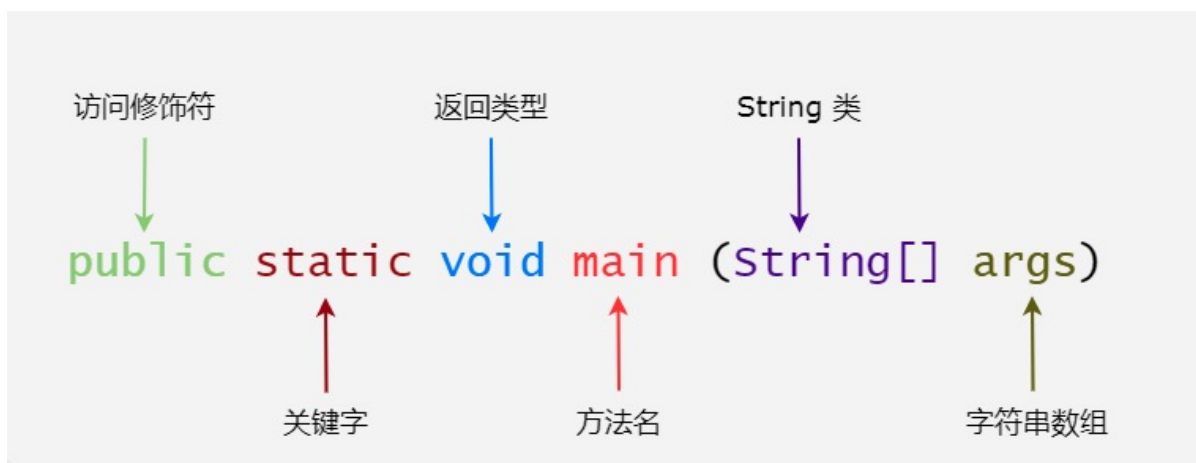
下面看一个简单的Java 程序，它将输出字符串 *Hello World*

文件名: HelloWorld.java

```
public class HelloWorld {  
    /* 第一个Java程序  
     * 它将输出字符串 Hello world  
     */  
    public static void main(String[] args) {  
        System.out.println("Hello world"); // 输出 Hello world  
    }  
}
```

我们可以分析一下这段代码：

1. **文件名**：源文件名必须和类名相同。当保存文件的时候，你应该使用类名作为文件名保存（切记Java是**大小写敏感**的，也就是说hello和Hello是不同的），文件名的后缀为 **.java**。（如果文件名和类名不相同则会导致编译错误）。
2. **类名**：对于所有的类来说，类名的首字母应该大写。如果类名由若干单词组成，那么每个单词的首字母应该大写（大驼峰命名），例如 **MyFirstJavaClass**。
3. **方法名**：所有的方法名都应该以小写字母开头。如果方法名含有若干单词，则后面的每个单词首字母大写（小驼峰命名），例如**myFirstMethod**。
4. **主方法入口**：所有的Java 程序由 **public static void main(String[] args)** 方法开始执行。



变量

数据类型

java内置的数据类型用很多种，可以大致分为几大类：整数，小数，字符，布尔值

1. **整数**：包括byte, short, int, long。可储存的数字大小依次增加，相应地，占用的内存依次增多。自己编写程序时往往不会考虑内存问题，通常用int作为整数类型。
2. **小数**：包括float, double。精度依次增加，相应地，占用的内存依次增多。自己写程序时，通常用double作为小数类型。
3. **字符**：包括char, String(严格来说，String不算是基本数据类型)。char储存一个单一的 16 位 Unicode 字符。String是任意长度的字符串（但是它实际上属于对象）。通常用String用作字符串变量。
4. **布尔值**：boolean。只有两个取值：true 和 false。

变量声明

```
type identifier [ = value][, identifier [= value] ...] ;
```

格式说明：

- type -- 数据类型。
- identifier -- 是变量名，可以使用逗号，隔开来声明多个同类型变量。

在声明一个变量时，可以为它加上初始值，也可以不加（不加初始值默认为0, null, false等）。

```
int a = 3; //声明变量a，类型为int，初始值为3
double b; //声明变量b，类型为double，不赋予初始值
```

想要修改变量时，不需要再写数据类型

```
a = 4; //把变量a修改为4
```

常量声明：

在语句最前面加上**final**关键词，通常与**static**关键词一起使用（表示静态）

```
static final int MAX_SIZE = 100;
```

注意常量命名时用**全大写字母**（虽然不这么做也不会报错，但是出于代码规范，强烈建议全大写）

还有参数变量，静态变量，局部变量等概念，这里不再赘述

循环

while

```
while( 布尔表达式 ) {
    //循环内容
}
```

只要布尔表达式为 true，循环就会一直执行下去。

例如这个累加的代码

```
public class Test {
    public static void main(String[] args) {
        int x = 10;
        while( x < 20 ) {
            System.out.print("value of x : " + x );
            x++;
            System.out.print("\n");
        }
    }
}
```

以上实例编译运行结果如下：

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

do...while

对于 while 语句而言，如果不满足条件，则不能进入循环。但有时候我们需要即使不满足条件，也至少执行一次。

do...while 循环和 while 循环相似，不同的是，do...while 循环至少会执行一次。

```
do {
    //代码语句
}while(布尔表达式);
```

注意：布尔表达式在循环体的后面，所以语句块在检测布尔表达式之前已经执行了。如果布尔表达式的值为 true，则语句块一直执行，直到布尔表达式的值为 false。

for循环

for循环执行的次数是在执行前就确定的。语法格式如下：

```
for(初始化； 布尔表达式； 更新) {
    //代码语句
}
```

关于 for 循环有以下几点说明：

- 最先执行初始化步骤。可以声明一种类型，但可初始化一个或多个循环控制变量，也可以是空语句。

- 然后，检测布尔表达式的值。如果为 true，循环体被执行。如果为false，循环终止，开始执行循环体后面的语句。
- 执行一次循环后，更新循环控制变量。
- 再次检测布尔表达式。循环执行上面的过程。

以下是一个常见的for循环，代表循环10次

```
public class Test {  
    public static void main(String[] args) {  
        for(int x = 0; x < 10; x++) {  
            System.out.print("value of x : " + x );  
            System.out.print("\n");  
        }  
    }  
}
```

输出：

```
value of x : 0  
value of x : 1  
value of x : 2  
value of x : 3  
value of x : 4  
value of x : 5  
value of x : 6  
value of x : 7  
value of x : 8  
value of x : 9
```

for循环还有一张加强表达式，用于遍历可迭代对象（如数组）

```
for(声明语句 : 表达式){  
    //代码句子  
}
```

以下是一个例子，循环遍历了一个数组

```
public class Test {  
    public static void main(String[] args){  
        int [] numbers = {10, 20, 30, 40, 50};  
  
        for(int x : numbers ){  
            System.out.print( x );  
            System.out.print(",");  
        }  
    }  
}
```

输出

```
10,20,30,40,50,
```

break与continue

break 主要用在循环语句或者 switch 语句中，用来跳出整个语句块。break 的用法很简单，就是循环结构中的一条语句：

```
break;
```

```
public class Test {
    public static void main(String[] args) {
        int [] numbers = {10, 20, 30, 40, 50};

        for(int x : numbers ) {
            // x 等于 30 时跳出循环
            if( x == 30 ) {
                break;
            }
            System.out.print( x );
            System.out.print("\n");
        }
    }
}
```

continue 适用于任何循环控制结构中。作用是让程序立刻跳转到下一次循环的迭代。

在 for 循环中，continue 语句使程序立即跳转到更新语句。

在 while 或者 do...while 循环中，程序立即跳转到布尔表达式的判断语句。

```
public class Test {
    public static void main(String[] args) {
        int [] numbers = {10, 20, 30, 40, 50};

        for(int x : numbers ) {
            // 当x=30时，直接跳到下一次循环
            if( x == 30 ) {
                continue;
            }
            System.out.print( x );
            System.out.print("\n");
        }
    }
}
```

条件

If

```
if(布尔表达式){
    //如果布尔表达式为true将执行的语句
}
```

if 语句后面可以跟 else 语句，当 if 语句的布尔表达式值为 false 时，else 语句块会被执行。

```
if(布尔表达式){  
    //如果布尔表达式的值为true  
}else{  
    //如果布尔表达式的值为false  
}
```

当然，也有else if

注意，若先前的判断有true，则不会执行后面的else if判断。else只会在先前所以判断都为false时执行。

以下是一个简单的例子

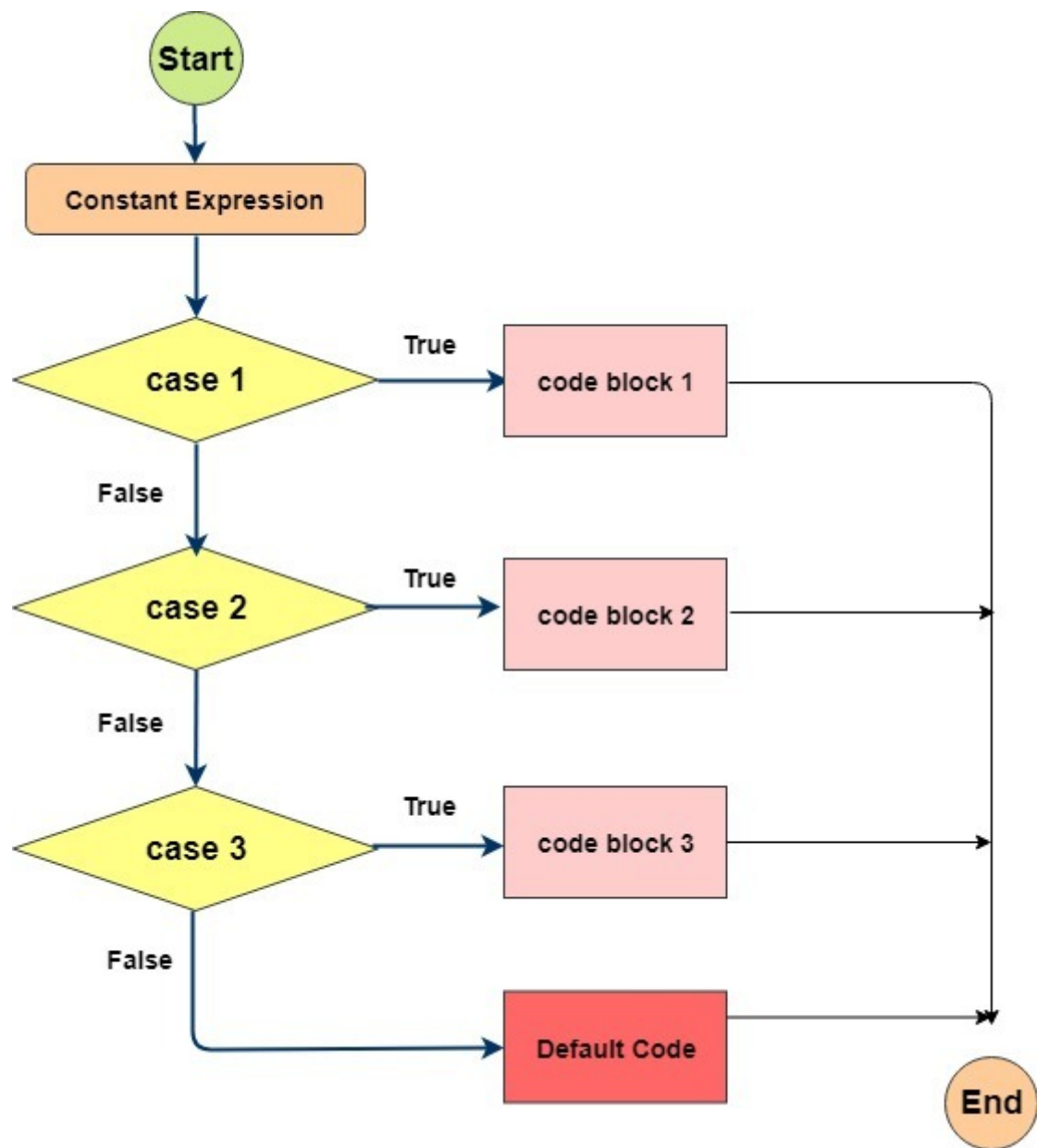
```
public class Test {  
    public static void main(String args[]){  
        int x = 30;  
  
        if( x == 10 ){  
            System.out.print("value of x is 10");  
        }else if( x == 20 ){  
            System.out.print("value of x is 20");  
        }else if( x == 30 ){  
            System.out.print("value of x is 30");  
        }else{  
            System.out.print("以上都不是");  
        }  
    }  
}
```

Switch case

switch case 语句判断一个变量与一系列值中某个值是否相等，每个值称为一个分支。

```
switch(expression){  
    case value :  
        //语句  
        break; //可选  
    case value :  
        //语句  
        break; //可选  
    //你可以有任意数量的case语句  
    default : //可选  
        //语句  
}
```

Switch case基本原理如下



- switch 语句可以拥有多个 case 语句。每个 case 后面跟一个要比较的值和冒号。
- case 语句中的值的数据类型必须与变量的数据类型相同，而且只能是常量或者字面常量。
- 当变量的值与 case 语句的值相等时，那么 case 语句之后的语句开始执行，直到 break 语句出现才会跳出 switch 语句。
- 当遇到 break 语句时，switch 语句终止。程序跳转到 switch 语句后面的语句执行。case 语句不必须要包含 break 语句。如果没有 break 语句出现，程序会继续执行下一条 case 语句，直到出现 break 语句。
- switch 语句可以包含一个 default 分支，该分支一般是 switch 语句的最后一个分支（可以在任何位置，但建议在最后一个）。default 在没有 case 语句的值和变量值相等的时候执行。default 分支不需要 break 语句。

switch case 执行时，一定会先进行匹配，匹配成功返回当前 case 的值，再根据是否有 break，判断是否继续输出，或是跳出判断。

以下是一个例子

```
public class Test {  
    public static void main(String args[]){  
        char grade = 'C';
```

```

switch(grade){
    case 'A' :
        System.out.println("优秀");
        break;
    case 'B' :
    case 'C' :
        System.out.println("良好");
        break;
    case 'D' :
        System.out.println("及格");
        break;
    case 'F' :
        System.out.println("你需要再努力努力");
        break;
    default :
        System.out.println("未知等级");
}
System.out.println("你的等级是 " + grade);
}
}

```

以上代码编译运行结果如下：

```

良好
你的等级是 C

```

如果 case 语句块中没有 break 语句时，匹配成功后，从当前 case 开始，后续所有 case 的值都会输出。

```

public class Test {
    public static void main(String args[]){
        int i = 1;
        switch(i){
            case 0:
                System.out.println("0");
            case 1:
                System.out.println("1");
            case 2:
                System.out.println("2");
            default:
                System.out.println("default");
        }
    }
}

```

以上代码编译运行结果如下：

```

1
2
default

```

重点：

- 除非有特意设计，一定要记得在case末尾加上**break**;

- 无论是否有default的处理逻辑，**都建议加上default分支**，哪怕里面是空的